

Deep Learning

Best Practices for the Real World



Prof. Anxiao (Andrew) Jiang

Learning Objectives:

1. Understand hyperparameter tuning
2. Understand mixed-precision training
3. Understand how to train Keras on multiple GPUs



Roadmap of this lecture:

1. hyperparameter tuning

2. Mixed-precision training

3. Train Keras on multiple GPUs



Hyper-parameter Optimization

How many layers in the model?

How many neurons or filters in each layer?

Which activation function to use?

Should Batch Normalization be used after a layer?

How much dropout to use?

And so on.

Use **KerasTuner** for Hyper-parameter Optimization

Let's start by installing KerasTuner:

```
!pip install keras-tuner -q
```

KerasTuner lets you replace hard-coded hyperparameter values, such as `units=32`, with a range of possible choices, such as `Int(name="units", min_value=16, max_value=64, step=16)`. This set of choices in a given model is called the *search space* of the hyperparameter tuning process.

To specify a search space, define a model-building function (see the next listing). It takes an `hp` argument, from which you can sample hyperparameter ranges, and it returns a compiled Keras model.

Listing 13.1 A KerasTuner model-building function

```
from tensorflow import keras
from tensorflow.keras import layers
```

Sample hyperparameter values from the `hp` object. After sampling, these values (such as the "units" variable here) are just regular Python constants.

```
def build_model(hp):
```

```
    units = hp.Int(name="units", min_value=16, max_value=64, step=16)
```

```
    model = keras.Sequential([
        layers.Dense(units, activation="relu"),
        layers.Dense(10, activation="softmax")
    ])
```

Different kinds of hyperparameters are available: `Int`, `Float`, `Boolean`, `Choice`.

```
    optimizer = hp.Choice(name="optimizer", values=["rmsprop", "adam"])
```

```
    model.compile(
```

```
        optimizer=optimizer,
```

```
        loss="sparse_categorical_crossentropy",
```

```
        metrics=["accuracy"])
```

```
    return model
```

The function returns a compiled model.

The next step is to define a “**tuner**.” Schematically, you can think of a tuner as a for loop that will repeatedly

- Pick a set of hyperparameter values
- Call the model-building function with these values to create a model
- Train the model and record its metrics

KerasTuner has several built-in tuners available—RandomSearch, BayesianOptimization, and Hyperband. Let’s try **BayesianOptimization**, a tuner that attempts to make smart predictions for which new hyperparameter values are likely to perform best given the outcomes of previous choices:

Specify the model-building function (or hyper-model instance).

```
tuner = kt.BayesianOptimization(  
    build_model,  
    objective="val_accuracy",  
    max_trials=100,
```

Specify the metric that the tuner will seek to optimize. Always specify validation metrics, since the goal of the search process is to find models that generalize!

Maximum number of different model configurations ("trials") to try before ending the search.

```
    executions_per_trial=2,  
    directory="mnist_kt_test",  
    overwrite=True,  
)
```

To reduce metrics variance, you can train the same model multiple times and average the results. `executions_per_trial` is how many training rounds (executions) to run for each model configuration (trial).

Where to store search logs

Whether to overwrite data in directory to start a new search. Set this to True if you've modified the model-building function, or to False to resume a previously started search with the same model-building function.

You can display an overview of the search space via `search_space_summary()`:

```
>>> tuner.search_space_summary()
Search space summary
Default search space size: 2
units (Int)
{"default": None,
 "conditions": [],
 "min_value": 16 ,
 "max_value": 64 ,
 "step": 16 ,
 "sampling": None}
optimizer (Choice)
{"default": "rmsprop",
 "conditions": [],
 "values": ["rmsprop", "adam"],
 "ordered": False}
```

Objective maximization and minimization

For built-in metrics (like accuracy, in our case), the *direction* of the metric (accuracy should be maximized, but a loss should be minimized) is inferred by KerasTuner. However, for a custom metric, you should specify it yourself, like this:

```
objective = kt.Objective(  
    name="val_accuracy",  
    direction="max")
```

```
tuner = kt.BayesianOptimization(  
    build_model,  
    objective=objective,  
    ...  
)
```

← The metric's name, as
found in epoch logs

← The metric's
desired direction:
"min" or "max"

Finally, let's launch the search. Don't forget to pass validation data, and make sure not to use your test set as validation data—otherwise you'd quickly start overfitting to your test data, and you wouldn't be able to trust your test metrics anymore:

```
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
x_train = x_train.reshape((-1, 28 * 28)).astype("float32") / 255
x_test = x_test.reshape((-1, 28 * 28)).astype("float32") / 255
x_train_full = x_train[:]
y_train_full = y_train[:]
```

Reserve these for later.

**Set these
aside as a
validation set.**

```
num_val_samples = 10000
x_train, x_val = x_train[:-num_val_samples], x_train[-num_val_samples:]
y_train, y_val = y_train[:-num_val_samples], y_train[-num_val_samples:]
callbacks = [
    > keras.callbacks.EarlyStopping(monitor="val_loss", patience=5),
]
```

```
tuner.search(
    x_train, y_train,
    batch_size=128,
    > epochs=100,
    validation_data=(x_val, y_val),
    callbacks=callbacks,
    verbose=2,
)
```

**This takes the same
arguments as fit()
(it simply passes them
down to fit() for each
new model).**

Use a large number of epochs (you don't know in advance how many epochs each model will need), and use an EarlyStopping callback to stop training when you start overfitting.

Quiz questions:

1. What is hyper-parameter tuning?
2. How to use KerasTuner to tune hyper-parameters?

Roadmap of this lecture:

1. hyperparameter tuning

2. Mixed-precision training

3. Train Keras on multiple GPUs



Speed up training on GPU with **mixed precision**

There are three of levels of precision you'd typically use:

- Half precision, or `float16`, where numbers are stored on 16 bits
- Single precision, or `float32`, where numbers are stored on 32 bits
- Double precision, or `float64`, where numbers are stored on 64 bits

Beware of dtype defaults

Single precision is the default floating-point type throughout Keras and TensorFlow: any tensor or variable you create will be in `float32` unless you specify otherwise. For NumPy arrays, however, the default is `float64`!

Converting a default NumPy array to a TensorFlow tensor will result in a `float64` tensor, which may not be what you want:

```
>>> import tensorflow as tf
>>> import numpy as np
>>> np_array = np.zeros((2, 2))
>>> tf_tensor = tf.convert_to_tensor(np_array)
>>> tf_tensor.dtype
tf.float64
```

Remember to be explicit about data types when converting NumPy arrays:

```
>>> np_array = np.zeros((2, 2))
>>> tf_tensor = tf.convert_to_tensor(np_array, dtype="float32")
>>> tf_tensor.dtype
tf.float32
```

Specify the dtype explicitly.



Note that when you call the Keras `fit()` method with NumPy data, it will do this conversion for you.

MIXED-PRECISION TRAINING IN PRACTICE

When training on a GPU, you can turn on mixed precision like this:

```
from tensorflow import keras  
keras.mixed_precision.set_global_policy("mixed_float16")
```

Typically, most of the forward pass of the model will be done in float16 (with the exception of numerically unstable operations like softmax), while the weights of the model will be stored and updated in float32.

Keras layers have a `variable_dtype` and a `compute_dtype` attribute. By default, both of these are set to `float32`. When you turn on mixed precision, the `compute_dtype` of most layers switches to `float16`, and those layers will cast their inputs to `float16` and will perform their computations in `float16` (using half-precision copies of the weights). However, since their `variable_dtype` is still `float32`, their weights will be able to receive accurate `float32` updates from the optimizer, as opposed to half-precision updates.

Note that some operations may be numerically unstable in `float16` (in particular, `softmax` and `crossentropy`). If you need to opt out of mixed precision for a specific layer, just pass the argument `dtype="float32"` to the constructor of this layer.

Quiz questions:

1. What is mix-precision training?
2. How to do mix-precision training?

Roadmap of this lecture:

1. hyperparameter tuning

2. Mixed-precision training

3. Train Keras on multiple GPUs



Multi-GPU training

There are two ways to distribute computation across multiple devices: *data parallelism* and *model parallelism*.

With data parallelism, a single model is replicated on multiple devices or multiple machines. Each of the model replicas processes different batches of data, and then they merge their results.

With model parallelism, different parts of a single model run on different devices, processing a single batch of data together at the same time. This works best with models that have a naturally parallel architecture, such as models that feature multiple branches.

In practice, model parallelism is only used for models that are too large to fit on any single device: it isn't used as a way to speed up training of regular models, but as a way to train larger models. We won't cover model parallelism in these pages; instead we'll focus on what you'll be using most of the time: data parallelism. Let's take a look at how it works.

SINGLE-HOST, MULTI-DEVICE SYNCHRONOUS TRAINING

Once you're able to import tensorflow on a machine with multiple GPUs, you're seconds away from training a distributed model. It works like this:

```
strategy = tf.distribute.MirroredStrategy()
print(f"Number of devices: {strategy.num_replicas_in_sync}")
with strategy.scope():
    model = get_compiled_model()
    model.fit(
        train_dataset,
        epochs=100,
        validation_data=val_dataset,
        callbacks=callbacks)
```

Create a "distribution strategy" object. `MirroredStrategy` should be your go-to solution.

Everything that creates variables should be under the strategy scope. In general, this is only model construction and `compile()`.

Train the model on all available devices.

Use it to open a "strategy scope."

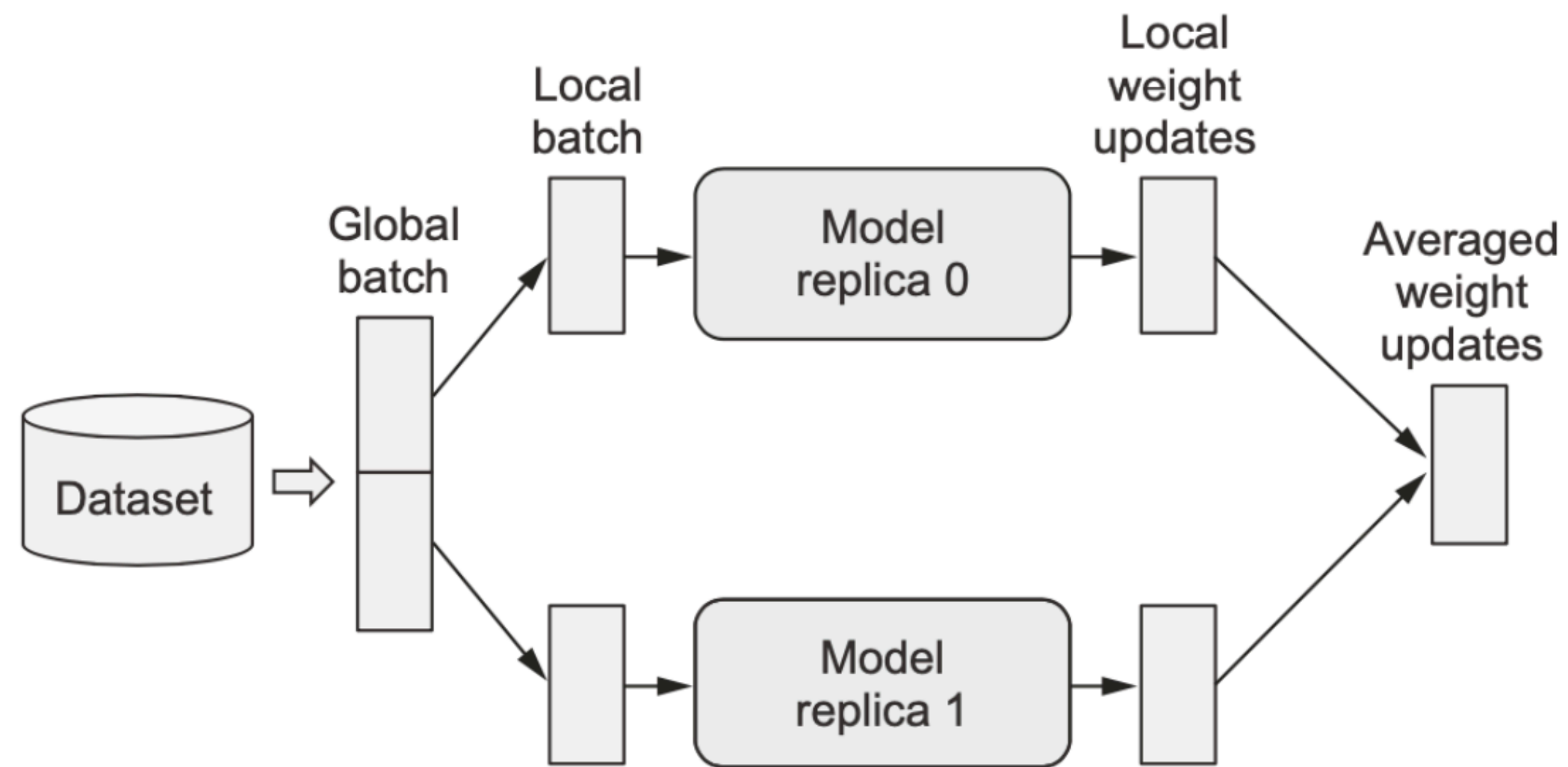


Figure 13.2 One step of MirroredStrategy training: each model replica computes local weight updates, which are then merged and used to update the state of all replicas.

Quiz question:

1. How to train a model on multiple GPUs?